

Data Structures and Algorithms

GÉZA HORVÁTH

Eleventh Lecture

Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 **Parallel algorithms: matrix multiplication, merge sort.**
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

Parallel algorithms – introduction

Parallel computers – computers with multiple processing units – have become increasingly common, and they span a wide range of prices and performance. Relatively inexpensive desktop and laptop chip multiprocessors contain a single multicore integrated-circuit chip that houses multiple processing "cores," each of which is a full-fledged processor that can access a **common memory**.

With the advent of multicore technology, however, every new laptop and desktop machine is now a shared-memory parallel computer, and the trend appears to be toward **shared-memory multiprocessing**.

Parallel algorithms – introduction

Until now, we have learned **serial algorithms**: suitable for running on a uniprocessor computer in which only one instruction executes at a time.

Today we are going to learn **parallel algorithms**: which can run on a multiprocessor computer that permits multiple instructions to execute concurrently. We use the model of **dynamic multithreaded algorithms**, which are amenable to algorithmic design and analysis, as well as to efficient implementation in practice.

Parallel algorithms – introduction

Dynamic multithreading allows programmers to specify parallelism in applications without worrying about communication protocols, load balancing, and other vagaries of static-thread programming. The concurrency platform contains a scheduler, which load-balances the computation automatically, thereby greatly simplifying the programmer's chore. Although the functionality of dynamic-multithreading environments is still evolving, almost all support two features: **nested parallelism** and **parallel loops**.

Nested parallelism allows a subroutine to be "spawned," allowing the caller to proceed while the spawned subroutine is computing its result.

A parallel loop is like an ordinary for loop, except that the iterations of the loop can execute concurrently.

New operations

Our model for dynamic multithreading is a simple extension of our serial programming model. We can describe a multithreaded algorithm by adding to our pseudocode just three "concurrency" keywords:

- **parallel**
- **spawn**
- **sync**

Moreover, if we delete these concurrency keywords from the multithreaded pseudocode, the resulting text is serial pseudocode for the same problem, which we call the "serialization" of the multithreaded algorithm.

Fibonacci numbers



Fibonacci numbers:

$$F_0 = 0 ,$$

$$F_1 = 1 ,$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i \geq 2 .$$

Fibonacci numbers – serial recursive algorithm

FIB(n)

```

1  if  $n \leq 1$ 
2      return  $n$ 
3  else  $x = \text{FIB}(n - 1)$ 
4       $y = \text{FIB}(n - 2)$ 
5      return  $x + y$ 

```

Fibonacci numbers:

$$F_0 = 0 ,$$

$$F_1 = 1 ,$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i \geq 2 .$$

Fibonacci numbers – number of steps

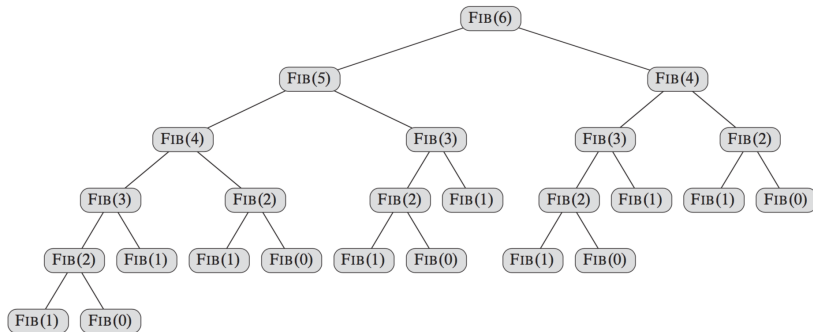


Figure 27.1 The tree of recursive procedure instances when computing FIB(6). Each instance of FIB with the same argument does the same work to produce the same result, providing an inefficient but interesting way to compute Fibonacci numbers.

Fibonacci numbers – number of steps

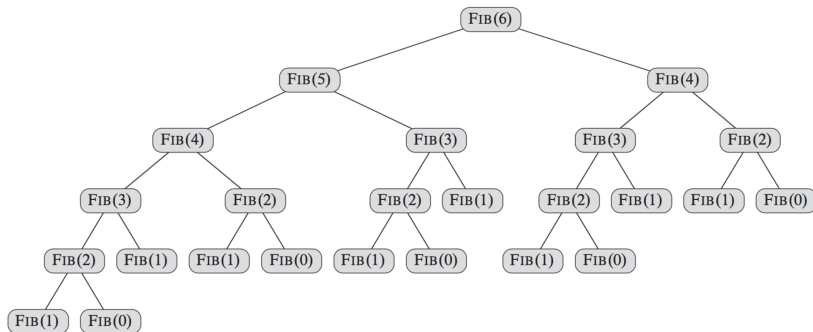


Figure 27.1 The tree of recursive procedure instances when computing FIB(6). Each instance of FIB with the same argument does the same work to produce the same result, providing an inefficient but interesting way to compute Fibonacci numbers.

$$T(n) = \Theta(\phi^n) ,$$

where $\phi = (1 + \sqrt{5})/2$ is the golden ratio

Fibonacci numbers – nested parallelism

P-FIB(n)

```

1  if  $n \leq 1$ 
2      return  $n$ 
3  else  $x = \text{spawn P-FIB}(n - 1)$ 
4       $y = \text{P-FIB}(n - 2)$ 
5      sync
6      return  $x + y$ 

```

Fibonacci numbers:

$$F_0 = 0 ,$$

$$F_1 = 1 ,$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i \geq 2 .$$

Nested parallelism occurs when the keyword **spawn** precedes a procedure call, as in line 3. The semantics of a **spawn** differs from an ordinary procedure call in that the procedure instance that executes the **spawn** – the **parent** – may continue to execute in parallel with the spawned subroutine – its **child** – instead of waiting for the child to complete, as would normally happen in a serial execution. In this case, while the spawned child is computing P-FIB($n-1$), the parent may go on to compute P-FIB($n-2$) in line 4 in parallel with the spawned child.

Fibonacci numbers – nested parallelism

P-FIB(n)

```

1  if  $n \leq 1$ 
2      return  $n$ 
3  else  $x = \text{spawn P-FIB}(n - 1)$ 
4       $y = \text{P-FIB}(n - 2)$ 
5      sync
6      return  $x + y$ 

```

Fibonacci numbers:

$$F_0 = 0,$$

$$F_1 = 1,$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i \geq 2.$$

A procedure cannot safely use the values returned by its spawned children until after it executes a sync statement, as in line 5. The keyword sync indicates that the procedure must wait as necessary for all its spawned children to complete before proceeding to the statement after the sync. In the P-FIB procedure, a sync is required before the return statement in line 6 to avoid the anomaly that would occur if x and y were summed before x was computed.

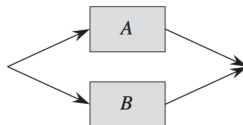
Multithreaded algorithms – work & span



$$\text{Work: } T_1(A \cup B) = T_1(A) + T_1(B)$$

$$\text{Span: } T_\infty(A \cup B) = T_\infty(A) + T_\infty(B)$$

(a)



$$\text{Work: } T_1(A \cup B) = T_1(A) + T_1(B)$$

$$\text{Span: } T_\infty(A \cup B) = \max(T_\infty(A), T_\infty(B))$$

(b)

Figure 27.3 The work and span of composed subcomputations. **(a)** When two subcomputations are joined in series, the work of the composition is the sum of their work, and the span of the composition is the sum of their spans. **(b)** When two subcomputations are joined in parallel, the work of the composition remains the sum of their work, but the span of the composition is only the maximum of their spans.

Multithreaded algorithms – running time

The actual running time of a multithreaded computation depends not only on its work and its span, but also on how many processors are available and how the scheduler allocates strands to processors. To denote the running time of a multithreaded computation on P processors, we shall subscript by P . For example, we might denote the running time of an algorithm on P processors by T_P . The work is the running time on a single processor, or T_1 . The span is the running time if we could run each strand on its own processor—in other words, if we had an unlimited number of processors—and so we denote the span by T_∞ .

express the span of P-FIB(n) as the recurrence

$$\begin{aligned} T_\infty(n) &= \max(T_\infty(n-1), T_\infty(n-2)) + \Theta(1) \\ &= T_\infty(n-1) + \Theta(1), \end{aligned}$$

which has solution $T_\infty(n) = \Theta(n)$

Parallel loops

As an example, consider the problem of multiplying an $n \times n$ matrix $A = (a_{ij})$ by an n -vector $x = (x_j)$. The resulting n -vector $y = (y_i)$ is given by the equation

$$y_i = \sum_{j=1}^n a_{ij} x_j ,$$

for $i = 1, 2, \dots, n$. We can perform matrix-vector multiplication by computing all the entries of y in parallel as follows:

MAT-VEC(A, x)

```

1   $n = A.rows$ 
2  let  $y$  be a new vector of length  $n$ 
3  parallel for  $i = 1$  to  $n$ 
4       $y_i = 0$ 
5  parallel for  $i = 1$  to  $n$ 
6      for  $j = 1$  to  $n$ 
7           $y_i = y_i + a_{ij} x_j$ 
8  return  $y$ 
```

Determinacy race

Definition

A determinacy race occurs when two logically parallel instructions access the same memory location and at least one of the instructions performs a write.

Example:

RACE-EXAMPLE()

```

1  x = 0
2  parallel for i = 1 to 2
3      x = x + 1
4  print x
    
```


Determinacy race

RACE-EXAMPLE()

```

1   $x = 0$ 
2  parallel for  $i = 1$  to 2
3       $x = x + 1$ 
4  print  $x$ 

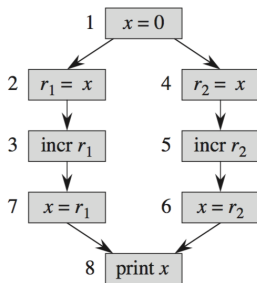
```

After initializing x to 0 in line 1, RACE-EXAMPLE creates two parallel strands, each of which increments x in line 3. Although it might seem that RACE-EXAMPLE should always print the value 2 (its serialization certainly does), it could instead print the value 1. Let's see how this anomaly might occur.

When a processor increments x , the operation is not indivisible, but is composed of a sequence of instructions:

1. Read x from memory into one of the processor's registers.
2. Increment the value in the register.
3. Write the value in the register back into x in memory.

Determinacy race



(a)

step	x	r_1	r_2
1	0	—	—
2	0	0	—
3	0	1	—
4	0	1	0
5	0	1	1
6	1	1	1
7	1	1	1

(b)

Figure 27.5 Illustration of the determinacy race in RACE-EXAMPLE. **(a)** A computation dag showing the dependencies among individual instructions. The processor registers are r_1 and r_2 . Instructions unrelated to the race, such as the implementation of loop control, are omitted. **(b)** An execution sequence that elicits the bug, showing the values of x in memory and registers r_1 and r_2 for each step in the execution sequence.

Determinacy race

MAT-VEC-WRONG(A, x)

```

1   $n = A.rows$ 
2  let  $y$  be a new vector of length  $n$ 
3  parallel for  $i = 1$  to  $n$ 
4       $y_i = 0$ 
5  parallel for  $i = 1$  to  $n$ 
6      parallel for  $j = 1$  to  $n$ 
7           $y_i = y_i + a_{ij}x_j$ 
8  return  $y$ 

```

This procedure is, unfortunately, incorrect due to races on updating y_i in line 7, which executes concurrently for all n values of j .

Multithreaded matrix multiplication

P-SQUARE-MATRIX-MULTIPLY(A, B)

```

1   $n = A.rows$ 
2  let  $C$  be a new  $n \times n$  matrix
3  parallel for  $i = 1$  to  $n$ 
4      parallel for  $j = 1$  to  $n$ 
5           $c_{ij} = 0$ 
6          for  $k = 1$  to  $n$ 
7               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
8  return  $C$ 

```

Original merge sort

MERGE-SORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \lfloor (p + r)/2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )

```

MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 

```

Multithreaded merge sort

MERGE-SORT'(A, p, r)

```

1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      spawn MERGE-SORT'(A, p, q)
4      MERGE-SORT'(A, q + 1, r)
5  sync
6      MERGE(A, p, q, r)
```

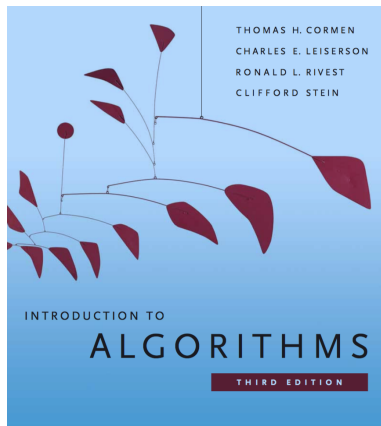
MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Further improvements can be achieved by applying multithreaded MERGE.

References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.